LINUX NETWORK TROUBLESHOOTING

OSI 7계층 장애 진단·대응 실습 명령어 총정리

상황별 장애 재현 · 진단 · 복구 명령어 모음 (L1~L7 + 시스템/컨테이너/클라우드/보안/자동화)

사용 방법

1. 0장의 network namespace 랩에서 장애를 일으키고 진단·복구를 반복해 익힙니다.
2. 실제 서버에서는 **읽기 전용 점검 명령**부터 사용하고, 설정 변경·방화벽 규칙 삽입은 반드시 영향도를 확인한 뒤 실행합니다.
3. 항상 **아래(L1)에서 위(L7)로** 확인하고, 이상 지점의 계층에서 원인을 좁힙니다.
4. <꺾쇠> 부분은 환경에 맞게 바꿔 쓰세요. 일부 명령은 별도 패키지 설치가 필요합니다 (ethtool, tcpdump, mtr, iperf3, nmap, dnsutils 등).

주의 nmap·부하·DDoS 방어 실험·보안 점검 명령은 **본인이 관리하거나 허가받은 시스템에만** 사용하세요.

목차

1. 0. 실습 랩 구성 (network namespace)
2. 1계층: 물리 계층 (Physical)
3. 2계층: 데이터 링크 계층 (Data Link)
4. 3계층: 네트워크 계층 (Network)
5. 4계층: 전송 계층 (Transport)
6. 5계층: 세션 계층 (Session)
7. 6계층: 표현 계층 (Presentation)
8. 7계층: 응용 계층 (Application)
9. 부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)
10. 부록 B. 패킷 캡처 · 분석 종합
11. 부록 C. 컨테이너 · 가상화 · 클라우드 환경
12. 부록 D. 보안 관점 점검 (침입·공격 징후)
13. 부록 E. 종합 진단 자동화 스크립트
14. 부록 F. 증상 → 계층 빠른 판별표
15. 부록 G. 계층별 핵심 명령어 한 장 요약

0. 실습 랩 구성 (network namespace)

운영 서버가 아닌 격리된 가상 환경에서 장애를 일으키고 진단·복구합니다. 모든 명령은 root 권한이 필요합니다.
C=client, S=server 네임스페이스에서 실행한다는 뜻입니다.

랩 생성: client ↔ server 를 veth 케이블로 연결

판단 ping 2회 성공하면 정상

```
ip netns add client; ip netns add server
ip link add veth-c type veth peer name veth-s
ip link set veth-c netns client; ip link set veth-s netns server
ip -n client addr add 10.0.0.1/24 dev veth-c
ip -n server addr add 10.0.0.2/24 dev veth-s
for n in client server; do ip -n $n link set lo up; done
ip -n client link set veth-c up; ip -n server link set veth-s up
alias C='ip netns exec client'; alias S='ip netns exec server'
C ping -c 2 10.0.0.2
```

간이 서비스 기동 (HTTP 8080)

판단 S ss -tlnp 에 8080 LISTEN

```
mkdir -p /tmp/www && echo ok > /tmp/www/index.html
S python3 -m http.server 8080 --bind 10.0.0.2 --directory /tmp/www &
C curl -s http://10.0.0.2:8080/
```

랩 초기화 / 정리

```
pkill -f "http.server 8080"; pkill -f s_server
ip netns del client; ip netns del server
rm -f /tmp/k.pem /tmp/c.pem
```

1계층: 물리 계층 (Physical)

케이블, 전원, NIC, 스위치 포트, 광모듈(SFP), 링크 협상. 증상: 링크 다운, 속도·듀플렉스 불일치, CRC 에러 증가.

1-1 인터페이스 DOWN / 케이블 분리

판단 state DOWN, NO-CARRIER, Link detected: no → 케이블·포트·관리자 down

```
S ip link set veth-s down            # 장애 발생
C ping -c 2 -W 1 10.0.0.2            # 실패
S ip -br link show veth-s            # DOWN
ip -n client link show veth-c        # NO-CARRIER
S ip link set veth-s up              # 복구
```

1-2 링크 상태·속도·듀플렉스 점검

판단 Speed 10/100Mb 또는 Half 협상 = 케이블/포트/오토네고 문제

```
ip -br link
ethtool eth0 | egrep "Speed|Duplex|Auto-negotiation|Link detected"
ethtool -S eth0 | egrep -i "err|drop|crc|fifo|miss"
ethtool -i eth0                      # 드라이버/펌웨어 버전
ethtool -a eth0                      # 흐름제어(pause)
ethtool -k eth0 | egrep "tso|gso|gro|rx-checksum"   # 오프로드
cat /sys/class/net/eth0/{operstate,carrier,speed,duplex}
cat /sys/class/net/eth0/carrier_changes   # 링크 플래핑 횟수
```

1-3 링크 플래핑 · NIC 하드웨어 오류 로그

판단 link is Down/Up 반복 = 케이블 불량, SFP 불량, 스위치 포트 오류

```
dmesg -T | egrep -i "eth0|link is (up|down)|nic|firmware|tx timeout|reset adapter"
journalctl -k --since "1 hour ago" | grep -i link
watch -n1 'cat /sys/class/net/eth0/carrier_changes'
ip monitor link                       # 링크 상태 변화 실시간 감시
```

1-4 속도·듀플렉스 강제 설정 / 오토네고 복구

판단 양단 설정이 달라 Duplex mismatch 가 나면 late collision, CRC 증가

```
ethtool -s eth0 autoneg off speed 1000 duplex full   # 강제(양단 동일해야 함)
ethtool -s eth0 autoneg on                           # 복구
ethtool -r eth0                                      # 오토네고 재시작
```

1-5 광모듈(SFP/SFP+) · 케이블 진단

판단 RX power 가 낮으면 광케이블/모듈 오염·불량

```
ethtool -m eth0                      # DOM: 온도, TX/RX power, 파장
ethtool --cable-test eth0            # 지원 NIC 의 TDR 케이블 테스트(드라이버 의존)
lspci | grep -i ethernet; lspci -vv -s <BDF> | grep -i -E "LnkSta|LnkCap"   # PCIe 링크 폭/속도
lshw -class network -short
```

1-6 하드웨어 모니터링 (전원·온도·디스크)

판단 서버 자체 물리 장애 확인

```
ipmitool sdr; ipmitool sel list      # BMC 센서 / 이벤트 로그
sensors                              # lm-sensors 온도
smartctl -a /dev/sda                 # 디스크 SMART
dmidecode -t system                  # 장비 정보
```

2계층: 데이터 링크 계층 (Data Link)

MAC, ARP, 스위치 학습, VLAN, 본딩, 브리지, STP. 같은 서브넷 내 통신 문제를 다룹니다.

2-1 잘못된 ARP 엔트리

판단 IP·라우팅 정상인데 통신 불가, MAC 이 실제와 다름

```
C ip neigh replace 10.0.0.2 lladdr 00:11:22:33:44:55 dev veth-c nud permanent
C ping -c 2 -W 1 10.0.0.2            # 실패
C ip neigh show; S ip link show veth-s
C tcpdump -i veth-c -n arp -c 5
C ip neigh del 10.0.0.2 dev veth-c   # 복구
C ip neigh flush all                 # 전체 캐시 삭제
```

2-2 ARP 상태 해석

판단 REACHABLE/STALE/DELAY/PROBE 정상, INCOMPLETE/FAILED = 응답 없음(L1·L2 문제)

```
ip neigh show
ip neigh show nud failed
ip neigh show nud incomplete
arp -n
arping -I eth0 -c 3 <대상IP>          # L2 도달성 (ICMP 와 무관)
cat /proc/net/arp
```

2-3 IP/MAC 중복 충돌

판단 arping -D 에서 응답이 오면 IP 중복

```
arping -D -I eth0 -c 3 <IP>          # DAD: 중복 IP 검사
arping -I eth0 -c 3 <IP>             # 응답한 MAC 목록 확인(2개 이상이면 충돌)
ip neigh show | sort -k5 | awk '{print $5}' | uniq -d    # 같은 MAC 이 여러 IP
# MAC 중복 실험
S ip link set veth-s down
S ip link set veth-s address $(C cat /sys/class/net/veth-c/address)
S ip link set veth-s up
```

2-4 패킷 손실 · 프레임 드롭 · 지연 주입

판단 tc netem 으로 장애를 재현하고 통계로 확인

```
S tc qdisc add dev veth-s root netem loss 30%
S tc qdisc change dev veth-s root netem delay 100ms 20ms loss 5% duplicate 1% corrupt 0.1% reorder 10%
C ping -c 20 10.0.0.2 | tail -3
S ip -s -s link show veth-s          # errors/dropped/overruns/carrier/collisions
S tc -s qdisc show dev veth-s
S tc qdisc del dev veth-s root       # 복구
nstat -az | egrep -i "InErrors|InDiscards|OutDiscards"
cat /proc/net/dev
```

2-5 L2 트래픽 캡처 (ARP, 브로드캐스트, STP)

판단 ARP 폭주·루프 징후 = 브로드캐스트 스톰

```
tcpdump -i eth0 -n -e arp
tcpdump -i eth0 -n -e ether broadcast -c 100
tcpdump -i eth0 -n -e ether multicast
tcpdump -i eth0 -n -e -c 20 stp      # STP BPDU
tcpdump -i eth0 -n -e vlan           # 802.1Q 태그 확인
tcpdump -i eth0 -n -e ether host aa:bb:cc:dd:ee:ff
```

2-6 VLAN 설정 · 점검

판단 VLAN ID 불일치 = 같은 스위치인데 통신 불가

```
ip link add link eth0 name eth0.100 type vlan id 100
ip addr add 192.168.100.10/24 dev eth0.100; ip link set eth0.100 up
ip -d link show eth0.100             # VLAN 상세
cat /proc/net/vlan/config
tcpdump -i eth0 -n -e vlan 100
ip link del eth0.100
```

2-7 본딩(bonding)/팀 상태 점검

판단 slave 하나가 down 이면 이중화 깨짐, mode 불일치(LACP) 주의

```
cat /proc/net/bonding/bond0          # Mode, MII Status, Slave 상태, LACP Partner
ip -d link show bond0
nmcli con show; nmcli dev status
ip link set eth1 down; ip link set eth1 up      # slave failover 시험
dmesg | grep -i bond
```

2-8 브리지 · 스위칭 점검 (가상환경/KVM/Docker)

판단 MAC 학습 테이블 / 포트 상태

```
bridge link show
bridge fdb show
bridge vlan show
brctl show; brctl showmacs br0
bridge -s link show
```

2-9 MTU / 점보프레임 불일치 (L2~L3 걸침)

판단 작은 패킷 OK, 큰 패킷 실패 = MTU 불일치

```
S ip link set veth-s mtu 1000
C ping -c 2 -s 100 10.0.0.2
C ping -c 2 -s 1400 -M do 10.0.0.2   # 실패(DF 설정)
C tracepath 10.0.0.2                 # pmtu
ip link show | grep mtu
S ip link set veth-s mtu 1500        # 복구
ping -M do -s 8972 <IP>              # 점보프레임(9000) 경로 검증
```

3계층: 네트워크 계층 (Network)

IP 주소, 서브넷, 게이트웨이, 라우팅, ICMP, NAT, 방화벽(L3), IPv6.

3-1 IP/서브넷 설정 오류

판단 Network is unreachable → 경로 없음. 서브넷 마스크 확인

```
C ip addr flush dev veth-c
C ip addr add 10.0.1.1/24 dev veth-c
C ping -c 2 -W 1 10.0.0.2            # Network is unreachable
C ip addr; C ip route
C ip addr flush dev veth-c; C ip addr add 10.0.0.1/24 dev veth-c   # 복구
```

3-2 게이트웨이/라우팅 누락

판단 ip route get 으로 실제 선택 경로를 확인

```
S ip addr add 192.168.100.1/24 dev lo
C ping -c 2 -W 1 192.168.100.1       # unreachable
C ip route get 192.168.100.1
C ip route add 192.168.100.0/24 via 10.0.0.2
C ip route add default via 10.0.0.2  # 기본 게이트웨이
C ip route del 192.168.100.0/24
ip route show table all
ip rule show                         # 정책 라우팅
ip route show table main
```

3-3 ICMP 만 차단 (ping 불가, 서비스 정상)

판단 ping 실패 ≠ 네트워크 장애

```
S iptables -A INPUT -p icmp -j DROP
C ping -c 2 -W 1 10.0.0.2
C curl -s -o /dev/null -w "%{http_code}\n" http://10.0.0.2:8080    # 200
S iptables -D INPUT -p icmp -j DROP
```

3-4 경로 추적 · 병목 구간 찾기

판단 * * * 는 ICMP 차단일 수 있음. mtr 로 구간별 손실 확인

```
traceroute -n 8.8.8.8
traceroute -T -p 443 -n <IP>         # TCP 로 추적(ICMP 차단 우회)
traceroute -I -n <IP>                # ICMP 모드
tracepath -n <IP>
mtr -rwzbc 100 8.8.8.8               # 리포트 모드, 손실·지연 통계
ping -c 100 -i 0.2 <IP> | tail -3
```

3-5 비대칭 라우팅 · rp_filter 로 인한 드롭

판단 여러 NIC 에서 응답이 다른 NIC 로 나가면 rp_filter 가 차단

```
sysctl net.ipv4.conf.all.rp_filter net.ipv4.conf.eth0.rp_filter
sysctl -w net.ipv4.conf.all.rp_filter=2     # loose 모드
nstat -az | grep -i IPReversePathFilter
sysctl -w net.ipv4.conf.all.log_martians=1; dmesg | grep -i martian
ip route get <외부IP> from <내IP>
```

3-6 IP 포워딩 · NAT (라우터/게이트웨이 역할)

판단 포워딩 꺼짐 또는 MASQUERADE 누락

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
```

```
iptables -t nat -L -n -v
iptables -t nat -A POSTROUTING -s 10.0.0.0/24 -o eth0 -j MASQUERADE
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
conntrack -L | head; conntrack -S
```

3-7 IPv6 점검

판단 IPv6 우선 선택으로 지연이 생기는 경우도 많음

```
ip -6 addr; ip -6 route; ip -6 neigh
ping -6 -c 3 ::1
ping -6 -c 3 2001:4860:4860::8888
traceroute6 -n 2001:4860:4860::8888
curl -4 -sI https://example.com | head -1; curl -6 -sI https://example.com | head -1
sysctl net.ipv6.conf.all.disable_ipv6
rdisc6 eth0                          # Router Advertisement 확인
```

3-8 DHCP · 주소 할당 문제

판단 169.254.x.x 또는 주소 없음 = DHCP 응답 없음

```
dhclient -v eth0                     # 요청 과정 출력
dhclient -r eth0                     # 반납
tcpdump -i eth0 -n port 67 or port 68
journalctl -u NetworkManager | grep -i dhcp
nmcli device show eth0 | grep -i -E "ip4|gateway|dns"
cat /var/lib/dhcp/dhclient.leases
networkctl status eth0
```

3-9 ICMP 에러 메시지 해석

판단 원인 분류

```
# Destination Net Unreachable: 경로 없음 / Host Unreachable: ARP 실패
# Frag needed (type3 code4): MTU / TTL exceeded: 루프 또는 홉 초과
tcpdump -i eth0 -n icmp
tcpdump -i eth0 -n 'icmp[icmptype] != icmp-echo and icmp[icmptype] != icmp-echoreply'
ping -t 1 8.8.8.8                    # TTL=1 로 첫 홉 확인(ttl 지정)
```

4계층: 전송 계층 (Transport)

TCP/UDP 포트, 3-way handshake, 방화벽(L4), 소켓 상태, 포트/연결 고갈, 재전송.

4-1 포트 미리슨 (Connection refused)

판단 즉시 refused = 리스너 없음 또는 REJECT

```
C nc -zv -w 3 10.0.0.2 9999         # refused
S ss -tulnp | grep 9999
S python3 -m http.server 9999 --bind 10.0.0.2 &
C nc -zv -w 3 10.0.0.2 9999         # succeeded
ss -tlnp; ss -ulnp; ss -lntp sport = :80
lsof -i :80; fuser -v 80/tcp
```

4-2 방화벽 DROP vs REJECT

판단 타임아웃 = DROP/경로 문제, 즉시 거부 = REJECT/리스너 없음

```
S iptables -A INPUT -p tcp --dport 8080 -j DROP
time C nc -zv -w 5 10.0.0.2 8080    # 5초 후 timed out
S iptables -D INPUT -p tcp --dport 8080 -j DROP
S iptables -A INPUT -p tcp --dport 8080 -j REJECT --reject-with tcp-reset
time C nc -zv -w 5 10.0.0.2 8080    # 즉시 refused
S iptables -D INPUT -p tcp --dport 8080 -j REJECT --reject-with tcp-reset
```

4-3 방화벽 정책 확인 (iptables / nftables / firewalld / ufw)

판단 규칙 순서·기본 정책(policy) 확인

```
iptables -L -v -n --line-numbers; iptables -S
iptables -t nat -L -n -v
nft list ruleset
firewall-cmd --state; firewall-cmd --list-all
firewall-cmd --permanent --add-port=8080/tcp && firewall-cmd --reload
ufw status verbose
iptables -I INPUT 1 -p tcp --dport 8080 -j ACCEPT   # 임시 허용
iptables -A INPUT -j LOG --log-prefix "DROP: "      # 드롭 로깅
journalctl -k | grep "DROP: "
```

4-4 3-Way Handshake 관찰

판단 SYN 만 보이고 SYN-ACK 없으면 서버/방화벽 문제, SYN-ACK 는 있는데 ACK 없으면 반대 방향 문제

```
S tcpdump -i veth-s -n 'tcp port 8080' -c 10
C curl -s -o /dev/null http://10.0.0.2:8080
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0'
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-rst != 0'      # RST 추적
tcpdump -i eth0 -nn 'tcp[tcpflags] == tcp-syn'          # SYN 만
tcpdump -i eth0 -nn 'tcp[tcpflags] & (tcp-syn|tcp-ack) == (tcp-syn|tcp-ack)'  # SYN-ACK
```

4-5 소켓 상태 분석

판단 TIME_WAIT 다수=정상 범위 점검, CLOSE_WAIT 다수=앱이 close 안 함, SYN_RECV 다수=SYN flood 의심

```
ss -s
ss -tan | awk 'NR>1{print $1}' | sort | uniq -c | sort -rn
ss -tan state time-wait | wc -l
ss -tan state close-wait | wc -l
ss -tan state syn-recv | wc -l
ss -tanp state close-wait             # 어느 프로세스인가
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ss -tnoi dst 10.0.0.2                # rtt, cwnd, 재전송, 타이머
netstat -ant | awk '{print $6}' | sort | uniq -c
```

4-6 재전송 · 지연 · 처리량

판단 retrans 증가 = 손실/혼잡. 대역폭과 지연을 직접 측정

```
nstat -az | egrep "TcpRetransSegs|TcpExtTCPLostRetransmit|TcpExtTCPTimeouts|TcpExtTCPSynRetrans"
netstat -s | egrep -i "retrans|timeout|failed|overflow|drop"
ss -ti | grep -E "retrans|rtt"
S iperf3 -s &
C iperf3 -c 10.0.0.2 -t 10          # TCP 처리량
C iperf3 -c 10.0.0.2 -u -b 100M     # UDP 손실/지터
C iperf3 -c 10.0.0.2 -P 4 -R        # 병렬·역방향
```

4-7 포트·소켓 고갈 / conntrack 가득 참

판단 Cannot assign requested address, nf_conntrack: table full

```
sysctl net.ipv4.ip_local_port_range
ss -tan | wc -l
sysctl net.ipv4.tcp_tw_reuse net.ipv4.tcp_fin_timeout
cat /proc/sys/net/netfilter/nf_conntrack_{count,max}
dmesg | grep -i "nf_conntrack: table full"
sysctl -w net.netfilter.nf_conntrack_max=262144
ulimit -n; cat /proc/sys/fs/file-nr; cat /proc/sys/fs/file-max
ls /proc/<PID>/fd | wc -l            # 프로세스 FD 사용량
```

4-8 백로그 · SYN flood 점검

판단 리슨 큐 넘침 → 접속 지연/실패

```
ss -ltn                              # Recv-Q(현재 대기) / Send-Q(백로그 한도)
nstat -az | egrep "ListenOverflows|ListenDrops|TCPReqQFullDrop|TCPReqQFullDoCookies"
sysctl net.core.somaxconn net.ipv4.tcp_max_syn_backlog net.ipv4.tcp_syncookies
sysctl -w net.ipv4.tcp_syncookies=1
dmesg | grep -i "SYN flooding"
```

4-9 UDP 포트 점검

판단 UDP 는 응답이 없어 판단이 어려움 → ICMP port unreachable 로 판단

```
nc -zvu <IP> 53
ss -ulnp
nmap -sU -p 53,123,161 <IP>
tcpdump -i eth0 -n udp port 53
S nc -ul 5000 &
echo hi | C nc -u -w1 10.0.0.2 5000
```

4-10 포트 스캔 · 서비스 노출 확인

판단 내 서버가 외부에 무엇을 열고 있는지 (허가된 대상에만 사용)

```
nmap -sT -p- <IP>                    # 전체 TCP
nmap -sV -p 22,80,443 <IP>           # 서비스/버전
nmap -Pn -p 443 <IP>                 # ping 차단 환경
ss -tulnp                            # 로컬에서 직접 확인
nc -zv <IP> 20-25
```

5계층: 세션 계층 (Session)

세션 수립·유지·종료, 타임아웃, keepalive, NFS/RPC, 로그인 세션.

5-1 수립된 세션과 타이머

판단 timer:(keepalive,...) / (on,...) 로 재전송·유휴 확인

```
S python3 -m http.server 8080 --bind 10.0.0.2 &
C nc 10.0.0.2 8080 &
S ss -o state established '( sport = :8080 )'
S ss -tnoi state established
ss -tn state established | wc -l
```

5-2 세션 중간 단절 · 재전송 관찰

판단 응답 경로 DROP 시 재전송 타이머가 증가

```
S iptables -A OUTPUT -p tcp --sport 8080 -j DROP
echo "GET / HTTP/1.0" | C nc -w 2 10.0.0.2 8080
C ss -tno state established          # timer:(on,...,retrans) 확인
S iptables -D OUTPUT -p tcp --sport 8080 -j DROP
cat /proc/sys/net/ipv4/tcp_retries2
```

5-3 Keepalive · 유휴 세션 끊김 대응

판단 방화벽/NAT/LB 의 idle timeout 이 keepalive 주기보다 짧으면 세션 끊김

```
sysctl net.ipv4.tcp_keepalive_time net.ipv4.tcp_keepalive_intvl net.ipv4.tcp_keepalive_probes
sysctl -w net.ipv4.tcp_keepalive_time=300
ssh -o ServerAliveInterval=30 -o ServerAliveCountMax=3 user@host
grep -E "ClientAlive|TCPKeepAlive" /etc/ssh/sshd_config
ss -tnoe state established | grep keepalive
```

5-4 NFS / RPC 세션

판단 hard/soft 마운트, stale file handle, portmapper 문제

```
rpcinfo -p <NFS서버>
showmount -e <NFS서버>
mount | grep nfs; nfsstat -m; nfsstat -c
mount -t nfs -o vers=4,timeo=600,retrans=2 <서버>:/export /mnt
systemctl status nfs-server rpcbind
umount -f -l /mnt                   # 응답 없는 마운트 강제 해제
dmesg | grep -i -E "nfs|rpc"
exportfs -v
```

5-5 로그인 세션 관리

판단 누가 접속 중인지, 세션 정리

```
who -a; w; last -n 20; lastb | head
loginctl list-sessions; loginctl show-session <ID>
loginctl terminate-session <ID>
pkill -KILL -t pts/2
ss -tnp state established '( sport = :22 )'
tmux ls; screen -ls                 # 끊김 대비 세션 유지 도구
```

5-6 SMB / LDAP / DB 세션

판단 세션 기반 서비스의 연결 상태

```
smbstatus; smbclient -L //<서버> -N
ldapsearch -x -H ldap://<서버> -b "" -s base
```

```
mysql -e "SHOW PROCESSLIST; SHOW STATUS LIKE 'Threads_connected'"
psql -c "select state,count(*) from pg_stat_activity group by 1;"
ss -tn dport = :3306 | wc -l
```

6계층: 표현 계층 (Presentation)

TLS/SSL 인증서, 암호군, 프로토콜 버전, 문자 인코딩, 압축, 직렬화 형식.

6-1 TLS 서버 구성 및 핸드셰이크 확인

판단 인증서 체인·SAN·만료일 확인

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout /tmp/k.pem -out /tmp/c.pem -days 30 -subj "/CN=test.loc
al"
S openssl s_server -accept 8443 -cert /tmp/c.pem -key /tmp/k.pem -www &
C openssl s_client -connect 10.0.0.2:8443 -servername test.local </dev/null 2>&1 | head -30
C openssl s_client -connect 10.0.0.2:8443 </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -
dates -ext subjectAltName
C curl -vk https://10.0.0.2:8443/ 2>&1 | egrep "SSL|TLS|subject|expire|issuer"
```

6-2 인증서 만료 점검

판단 -checkend 는 종료 코드로 판정 (모니터링에 활용)

```
openssl x509 -in /tmp/c.pem -noout -enddate
openssl x509 -in /tmp/c.pem -noout -checkend 86400 && echo "24h 이상 유효" || echo "24h 내 만료"
openssl x509 -in /tmp/c.pem -noout -checkend 0 && echo "유효" || echo "만료됨"
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null | openssl x509 -noou
t -enddate
for h in a.com b.com; do echo -n "$h "; echo | openssl s_client -connect $h:443 -servername $h 2>/dev/nul
l | openssl x509 -noout -enddate; done
```

6-3 프로토콜/암호군 불일치

판단 handshake failure / no protocols available

```
C openssl s_client -connect 10.0.0.2:8443 -tls1_2 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect 10.0.0.2:8443 -tls1_3 </dev/null 2>&1 | grep -E "Protocol|Cipher"
C openssl s_client -connect 10.0.0.2:8443 -tls1_1 </dev/null 2>&1 | tail -5
openssl ciphers -v 'HIGH:!aNULL' | head
nmap --script ssl-enum-ciphers -p 443 <IP>
curl --tlsv1.2 --tls-max 1.2 -vI https://<호스트>
```

6-4 신뢰 체인 · 호스트명 불일치 · 시스템 CA

판단 (60) SSL certificate problem

```
C curl -v https://10.0.0.2:8443/ 2>&1 | grep -iE "certificate|verify|SSL"
C curl --cacert /tmp/c.pem --resolve test.local:8443:10.0.0.2 https://test.local:8443/ -o /dev/null -w "%
{http_code}\n"
openssl verify -CAfile ca.pem server.pem
openssl s_client -connect <호스트>:443 -showcerts </dev/null    # 중간 인증서 누락 확인
update-ca-certificates               # Debian/Ubuntu
update-ca-trust extract              # RHEL 계열
curl -v --cacert /etc/ssl/certs/ca-certificates.crt https://<호스트>
date; timedatectl                    # 시간 오차도 인증서 오류의 흔한 원인
```

6-5 SSH 호스트키 · 키 형식 문제

판단 REMOTE HOST IDENTIFICATION HAS CHANGED

```
ssh-keyscan -t ed25519,rsa <호스트>
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub     # 핑거프린트
ssh-keygen -R <호스트>               # known_hosts 에서 제거
ssh -vvv -o PreferredAuthentications=publickey user@host
ssh-keygen -y -f ~/.ssh/id_ed25519   # 개인키로 공개키 복원
ssh -Q cipher; ssh -Q kex            # 지원 알고리즘
```

6-6 문자셋/인코딩 깨짐

판단 한글이 ???, 깨짐 → locale/파일 인코딩 확인

```
locale; locale -a | egrep -i "utf|euc"
echo "안녕하세요" > /tmp/utf8.txt
iconv -f UTF-8 -t EUC-KR /tmp/utf8.txt > /tmp/euckr.txt
file -i /tmp/utf8.txt /tmp/euckr.txt
cat /tmp/euckr.txt                  # 깨짐
iconv -f EUC-KR -t UTF-8 /tmp/euckr.txt
localectl status; localectl set-locale LANG=ko_KR.UTF-8
export LANG=en_US.UTF-8 LC_ALL=en_US.UTF-8
convmv -f euc-kr -t utf-8 -r --notest .        # 파일명 인코딩 변환
curl -sI <URL> | grep -i content-type          # charset 확인
```

6-7 압축 · 직렬화 · 데이터 형식 검증

판단 Content-Encoding, JSON/XML 형식 오류

```
curl -sI -H "Accept-Encoding: gzip" <URL> | grep -i content-encoding
curl -s --compressed <URL> | head
echo '{"a":1' | python3 -m json.tool         # 문법 오류 확인
jq . file.json; xmllint --noout file.xml
gzip -t file.gz; tar -tzf file.tgz >/dev/null
base64 -d <<< "aGVsbG8="; file unknown.bin
```

7계층: 응용 계층 (Application)

HTTP, DNS, SSH, 메일, DB, 웹서버/프록시/LB, 서비스 프로세스와 로그.

7-1 HTTP 상태 코드 · 응답시간 분해

판단 느림의 원인을 DNS/연결/TLS/서버처리/전송으로 분리

```
C curl -sI http://10.0.0.2:8080/ | head -1        # 200
C curl -sI http://10.0.0.2:8080/nope | head -1     # 404
C curl -o /dev/null -s -w "dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} ttfb:%{time_
starttransfer} total:%{time_total} code:%{http_code}\n" http://10.0.0.2:8080/
curl -v https://<호스트>/ 2>&1 | head -40
curl -L -I https://<호스트>/                      # 리다이렉트 추적
curl -H "Host: a.example.com" http://<IP>/       # 가상호스트 테스트
curl -X POST -d '{"a":1}' -H 'Content-Type: application/json' <URL>
curl --max-time 5 --retry 3 <URL>
chmod 000 /tmp/www/index.html; C curl -sI http://10.0.0.2:8080/ | head -1; chmod 644 /tmp/www/index.html
```

7-2 상태 코드별 대응 가이드

판단 코드에 따른 1차 조치

```
# 301/302: 리다이렉트 루프 → curl -L -v 로 Location 확인
# 400: 요청 형식/헤더 크기 / 401·403: 인증·권한·ACL·SELinux
# 404: 경로·DocumentRoot / 408·504: 타임아웃(프록시↔업스트림)
# 413: 업로드 크기 제한(client_max_body_size) / 429: rate limit
# 500: 앱 오류 → 앱 로그 / 502: 업스트림 다운·응답 이상 / 503: 과부하·점검
getenforce; ausearch -m avc -ts recent             # SELinux 거부
ls -ldZ /var/www/html; restorecon -Rv /var/www/html
namei -l /var/www/html/index.html                  # 경로상 권한 확인
nginx -T | grep -E "client_max_body_size|proxy_read_timeout|upstream"
```

7-3 DNS 장애 진단

판단 ping 8.8.8.8 OK + ping 도메인 FAIL = DNS 문제

```
cat /etc/resolv.conf; resolvectl status | head -20
getent hosts example.com             # 시스템 리졸버 경로 그대로(nsswitch 포함)
dig example.com; dig +short example.com
dig @8.8.8.8 example.com             # 서버 비교
dig +trace example.com               # 루트부터 위임 추적
dig example.com MX / NS / TXT / AAAA / CNAME / SOA
dig -x 8.8.8.8                       # 역방향
dig +tcp example.com; dig +dnssec example.com
dig @10.255.255.1 example.com +time=2 +tries=1   # 타임아웃 재현
nslookup example.com 8.8.8.8
host -a example.com
grep hosts /etc/nsswitch.conf; cat /etc/hosts
resolvectl query example.com; resolvectl flush-caches
systemd-resolve --statistics
tcpdump -i eth0 -n port 53
time getent hosts example.com        # 해석 지연 측정
```

7-4 서비스 프로세스 (systemd)

판단 서비스 다운이면 Connection refused 로 4계층처럼 보임 → ss 로 리스너 확인

```
systemctl status nginx; systemctl is-active nginx; systemctl is-enabled nginx
systemctl --failed
systemctl restart nginx; systemctl reload nginx
journalctl -u nginx -n 50 --no-pager
journalctl -u nginx --since "10 min ago" -p err
journalctl -xe
nginx -t; apachectl configtest; sshd -t; named-checkconf; haproxy -c -f /etc/haproxy/haproxy.cfg
```

```
systemctl cat nginx; systemctl list-dependencies nginx
systemctl stop nginx; curl -sI http://localhost | head -1; ss -tlnp | grep ':80 '; systemctl start nginx
ps aux --sort=-%cpu | head; ps -eo pid,ppid,stat,cmd | grep -w Z   # 좀비
dmesg | grep -i -E "oom|killed process"
```

7-5 로그 분석

판단 상태코드 분포, 상위 IP, 에러 패턴, 시간대별 급증

```
tail -f /var/log/nginx/error.log
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
grep -c ' 5[0-9][0-9] ' /var/log/nginx/access.log
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
awk '{print substr($4,2,14)}' /var/log/nginx/access.log | uniq -c | sort -rn | head    # 분 단위 요청량
awk '$NF > 2 {print $7,$NF}' access.log | sort -k2 -nr | head                          # 느린 요청(응답시간
필드)
grep -iE "error|fail|denied|timeout|refused" /var/log/syslog | tail -20
journalctl -p err -b --no-pager | tail -30
journalctl --since "2026-11-11 00:00" --until "2026-11-11 01:00"
zgrep "ERROR" /var/log/app/*.gz
lnav /var/log/syslog                # 설치되어 있는 경우
```

7-6 SSH 접속 장애

판단 단계: 도달성 → 포트 → 핸드셰이크 → 인증

```
ssh -vvv user@host
nc -zv host 22; ss -tlnp | grep sshd
sshd -T | egrep "^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|allowgroup
s|maxauthtries)"
journalctl -u sshd -n 30 --no-pager; tail /var/log/auth.log /var/log/secure
lastb | head; last | head
ls -ld ~ ~/.ssh ~/.ssh/authorized_keys     # 권한: 700 / 600
chmod 700 ~/.ssh; chmod 600 ~/.ssh/authorized_keys
restorecon -Rv ~/.ssh                     # SELinux
ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 user@host true
fail2ban-client status sshd; fail2ban-client set sshd unbanip <IP>
ssh -J jump user@target; ssh -L 8080:localhost:80 user@host
```

7-7 웹서버 · 리버스 프록시 · LB 점검

판단 502/504 는 프록시와 업스트림 사이 문제

```
nginx -T | head -100; nginx -s reload
curl -sI http://127.0.0.1:<업스트림포트>/           # 업스트림 직접 호출
tail -f /var/log/nginx/error.log | grep -i upstream
apachectl -S; apachectl -t -D DUMP_MODULES
echo "show stat" | socat stdio /var/run/haproxy.sock | cut -d, -f1,2,18 | column -s, -t
ab -n 1000 -c 50 http://<호스트>/                  # 부하 테스트
wrk -t4 -c100 -d30s http://<호스트>/
curl -w "@curl-format.txt" -o /dev/null -s <URL>
```

7-8 DB · 메일 · 기타 서비스

판단 포트 → 인증 → 쿼리 순서로

```
mysqladmin -h <호스트> -u root -p ping; mysql -h <호스트> -e "select 1"
pg_isready -h <호스트> -p 5432
redis-cli -h <호스트> ping; redis-cli info clients
nc -zv <호스트> 25 587 993
openssl s_client -starttls smtp -connect <호스트>:587
swaks --to a@b.com --server <호스트>
postqueue -p; tail /var/log/mail.log
mongosh --host <호스트> --eval 'db.runCommand({ping:1})'
curl -s localhost:9200/_cluster/health?pretty      # Elasticsearch
ntpq -p; chronyc tracking; timedatectl             # 시간 동기화
```

7-9 시간 동기화 · 로그 시각 불일치 (자주 간과)

판단 인증서·Kerberos·로그 상관분석에 영향

```
timedatectl status
chronyc sources -v; chronyc tracking
ntpq -pn
hwclock --show
timedatectl set-ntp true
```

부록 A. 시스템 자원 병목 (네트워크처럼 보이는 장애)

CPU, 메모리, 디스크, 파일 디스크립터 문제가 네트워크 지연·타임아웃으로 나타나는 경우가 많습니다.

A-1 CPU · 부하 · 인터럽트

판단 softirq 가 한 코어에 몰리면 패킷 드롭

```
uptime; top -bn1 | head -15; mpstat -P ALL 1 3; vmstat 1 5
pidstat 1 3
cat /proc/softirqs | egrep "NET_RX|NET_TX"
cat /proc/interrupts | grep eth0
cat /proc/net/softnet_stat           # 두 번째 열(dropped) 증가 여부
ethtool -l eth0; ethtool -g eth0     # 큐 수 / 링버퍼 크기
ethtool -G eth0 rx 4096              # 링버퍼 확대
sysctl net.core.netdev_max_backlog net.core.rmem_max net.core.wmem_max
```

A-2 메모리 · OOM

판단 OOM 으로 서비스가 죽으면 갑자기 refused

```
free -h; vmstat 1 5; cat /proc/meminfo | head -20
ps aux --sort=-%mem | head
dmesg -T | grep -i -E "out of memory|oom-killer|killed process"
journalctl -k | grep -i oom
slabtop -o | head; sysctl vm.swappiness vm.overcommit_memory
smem -tk 2>/dev/null | tail
```

A-3 디스크 · I/O · inode

판단 디스크 가득 → 로그/세션/소켓 파일 생성 실패

```
df -h; df -i
du -xh / --max-depth=2 2>/dev/null | sort -rh | head
lsof +L1                             # 삭제됐지만 열려 있는 파일
iostat -xz 1 3; iotop -oPa
lsblk; mount | grep -E "ro,|errors"
dmesg | grep -i -E "i/o error|ext4|xfs|read-only"
```

A-4 프로세스 · 파일 디스크립터 · 스레드

판단 Too many open files

```
ulimit -a; cat /proc/<PID>/limits
ls /proc/<PID>/fd | wc -l
lsof -p <PID> | wc -l; lsof -i -nP | awk '{print $1}' | sort | uniq -c | sort -rn
cat /proc/sys/fs/file-nr
strace -f -p <PID> -e trace=network -tt   # 시스템콜 추적
perf top; ltrace -p <PID>
systemctl show nginx -p LimitNOFILE
```

A-5 커널 네트워크 파라미터 일괄 점검

판단 튜닝 전 현재값 기록

```
sysctl -a 2>/dev/null | egrep "net\.(core|ipv4\.tcp|ipv4\.ip_local|netfilter)" | head -80
sysctl net.ipv4.tcp_congestion_control net.core.default_qdisc
sysctl net.ipv4.tcp_rmem net.ipv4.tcp_wmem
sysctl -p /etc/sysctl.conf; sysctl --system
nstat -az | head -50; cat /proc/net/snmp; cat /proc/net/netstat
```

부록 B. 패킷 캡처 · 분석 종합

어느 계층 문제인지 불확실할 때 가장 확실한 방법은 패킷을 직접 보는 것입니다.

B-1 tcpdump 필터 모음

판단 캡처 후 Wireshark 로 분석

```
tcpdump -i any -nn host 10.0.0.2
tcpdump -i eth0 -nn port 443 and host <IP>
tcpdump -i eth0 -nn net 10.0.0.0/24
tcpdump -i eth0 -nn 'tcp port 80 and (tcp[tcpflags] & tcp-syn != 0)'
tcpdump -i eth0 -nn -A 'tcp port 80' -c 20          # ASCII 출력(평문 HTTP)
tcpdump -i eth0 -nn -XX -c 5                        # 헥사 덤프
tcpdump -i eth0 -nn not port 22                     # SSH 제외
tcpdump -i eth0 -nn 'icmp or arp'
tcpdump -i eth0 -nn -s0 -w /tmp/cap.pcap -C 100 -W 5   # 100MB x 5개 순환
tcpdump -r /tmp/cap.pcap -nn | head
tcpdump -i eth0 -nn 'ip[8] < 5'                     # TTL 낮은 패킷
tcpdump -i eth0 -nn 'greater 1400'                  # 큰 패킷
```

B-2 tshark / 기타 분석 도구

판단 통계·대화 단위 분석

```
tshark -r /tmp/cap.pcap -q -z conv,tcp
tshark -r /tmp/cap.pcap -Y "tcp.analysis.retransmission" | head
tshark -r /tmp/cap.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri
tshark -r /tmp/cap.pcap -Y "dns" -T fields -e dns.qry.name -e dns.flags.rcode
tshark -r /tmp/cap.pcap -q -z io,stat,1
tshark -r /tmp/cap.pcap -Y "tls.handshake.type==1" -T fields -e tls.handshake.extensions_server_name
ngrep -d eth0 -q "GET" port 80
iftop -i eth0; nload eth0; iptraf-ng
nethogs eth0                          # 프로세스별 대역폭
bmon; vnstat -l
```

B-3 실시간 트래픽 · 대역폭

판단 누가 대역을 쓰는가

```
sar -n DEV 1 5; sar -n TCP,ETCP 1 5; sar -n SOCK 1 3
ip -s link show eth0
watch -n1 "cat /proc/net/dev"
ss -tin | grep -E "bytes_sent|bytes_acked"
cat /sys/class/net/eth0/statistics/{rx_bytes,tx_bytes,rx_dropped,tx_dropped,rx_errors,tx_errors}
dstat -n 1 5
```

부록 C. 컨테이너 · 가상화 · 클라우드 환경

Docker/Kubernetes/클라우드에서는 L2~L4 가 오버레이·iptables·보안그룹으로 추상화됩니다.

C-1 Docker

판단 컨테이너 네트워크 확인

```
docker ps; docker network ls; docker network inspect bridge
docker inspect -f '{{.NetworkSettings.IPAddress}}' <컨테이너>
docker exec -it <컨테이너> sh -c "ip a; ip r; cat /etc/resolv.conf; ss -tlnp"
docker logs --tail 100 -f <컨테이너>
docker port <컨테이너>
nsenter -t $(docker inspect -f '{{.State.Pid}}' <컨테이너>) -n ss -tlnp   # 호스트 도구로 컨테이너 네트워크 확인
iptables -t nat -L DOCKER -n -v
docker stats --no-stream; docker events --since 10m
```

C-2 Kubernetes

판단 Service → Endpoint → Pod → DNS 순으로

```
kubectl get pods -A -o wide; kubectl describe pod <파드>
kubectl logs <파드> --previous; kubectl get events --sort-by=.lastTimestamp
kubectl get svc,ep,ingress -A
kubectl exec -it <파드> -- sh -c "nslookup kubernetes.default; curl -sI http://<svc>:<port>"
kubectl run tmp --rm -it --image=nicolaka/netshoot -- bash     # 디버그 파드
kubectl get networkpolicy -A
kubectl -n kube-system logs -l k8s-app=kube-dns
kubectl top nodes; kubectl top pods
kubectl port-forward svc/<svc> 8080:80
kubectl debug node/<노드> -it --image=busybox
```

C-3 클라우드 (AWS 예시)

판단 OS 내부 점검 + 보안그룹/NACL/라우팅 테이블

```
curl -s http://169.254.169.254/latest/meta-data/local-ipv4      # IMDS(v1 허용 시)
aws ec2 describe-security-groups --group-ids <sg-id>
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-id>
aws ec2 describe-network-acls
aws ec2 describe-instance-status --instance-ids <id>
aws ec2 get-console-output --instance-id <id>                   # 부팅 로그
# 4계층 도달 확인: OS 방화벽 + 보안그룹 + NACL 세 곳을 모두 점검
VPC Reachability Analyzer 사용(콘솔/CLI)
```

C-4 VPN · 터널 · 오버레이

판단 MTU 와 시계열 핸드셰이크 확인

```
wg show; ip -d link show wg0
ipsec statusall; ip xfrm state; ip xfrm policy
ip tunnel show; ip -d link show type vxlan
ping -M do -s 1372 <터널건너편IP>      # 터널 MTU 확인
openvpn --config client.ovpn --verb 4
```

부록 D. 보안 관점 점검 (침입·공격 징후)

DDoS, 무차별 대입, 의심 연결, 비인가 포트 등. 자신이 관리하는 시스템에만 사용하세요.

D-1 의심 연결 · 비인가 포트

판단 알 수 없는 리슨 포트/외부 연결 확인

```
ss -tulnp; ss -tnp state established
lsof -i -nP | grep -v 127.0.0.1
netstat -tunap | grep -v LISTEN | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
ps auxf; ls -la /tmp /var/tmp /dev/shm
crontab -l; ls /etc/cron.*; systemctl list-timers
find / -xdev -mtime -1 -type f 2>/dev/null | head
rkhunter --check; chkrootkit
```

D-2 무차별 대입 · 로그인 시도 분석

판단 공격 IP 추출 및 차단

```
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head
lastb | awk '{print $3}' | sort | uniq -c | sort -rn | head
journalctl -u sshd | grep -i "invalid user" | tail
iptables -I INPUT -s <공격IP> -j DROP
nft add rule inet filter input ip saddr <공격IP> drop
fail2ban-client status; fail2ban-client status sshd
ipset create block hash:ip; ipset add block <IP>; iptables -I INPUT -m set --match-set block src -j DROP
```

D-3 DDoS / 트래픽 폭주 대응

판단 SYN flood, 연결 수 제한, 레이트 리밋

```
ss -tan state syn-recv | wc -l
ss -tan | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn | head
sysctl -w net.ipv4.tcp_syncookies=1
iptables -A INPUT -p tcp --syn -m limit --limit 50/s --limit-burst 100 -j ACCEPT
iptables -A INPUT -p tcp --syn -j DROP
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 50 -j REJECT
iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/s -j ACCEPT
tcpdump -i eth0 -nn -c 1000 | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn | head
```

D-4 감사 · 무결성

판단 파일 변경과 접근 기록

```
auditctl -l; ausearch -ts today -m USER_LOGIN
aide --check; rpm -Va | head; debsums -c
last -F | head; lastlog | head
getent passwd | awk -F: '$3==0'      # UID 0 계정
awk -F: '($2=="") {print $1}' /etc/shadow   # 빈 비밀번호
```

부록 E. 종합 진단 자동화 스크립트

아래에서 위로(L1→L7) 점검하는 한 방 스크립트와, 로그를 수집해 보고서로 남기는 스크립트입니다.

E-1 diag.sh: 계층별 자동 점검

판단 사용: ./diag.sh <대상IP> <포트> [도메인]

```
#!/bin/bash
IF=${IF:-eth0}; GW=$(ip route | awk '/default/{print $3; exit}')
T=$1; P=$2; D=$3
echo "[L1] 링크:";   ip -br link show $IF; ethtool $IF 2>/dev/null | egrep "Link detected|Speed|Duplex"
echo "[L2] ARP:";    ip neigh show | head -5; ip -s link show $IF | grep -A1 -E "RX:|TX:"
echo "[L3] IP/GW:";  ip -br addr show $IF; ip route show default
echo -n "[L3] GW ping: ";  ping -c 2 -W 1 $GW >/dev/null && echo OK || echo FAIL
echo -n "[L3] 대상 ping: "; ping -c 2 -W 1 $T >/dev/null && echo OK || echo "FAIL (ICMP 차단 가능성)"
echo -n "[L4] 포트 $P: ";   nc -zv -w 3 $T $P 2>&1 | tail -1
[ -n "$D" ] && { echo "[L7] DNS:"; dig +short $D | head -3; }
[ "$P" = "443" ] && { echo "[L6] TLS:"; echo | openssl s_client -connect $T:443 ${D:+-servername $D} 2>/d
ev/null | openssl x509 -noout -dates; }
echo "[L7] HTTP:";   curl -sI -m 5 ${D:+--resolve $D:$P:$T} "http://${D:-$T}:$P/" | head -1
```

E-2 장애 증거 일괄 수집

판단 장애 발생 직후 실행해 두면 사후 분석에 유용

```
#!/bin/bash
OUT=/tmp/incident-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $OUT
{ date; uptime; ip -br addr; ip route; ip neigh; ip -s link; ss -s; ss -tanp; } > $OUT/net.txt 2>&1
{ free -h; df -h; df -i; vmstat 1 3; top -bn1 | head -30; } > $OUT/res.txt 2>&1
{ nstat -az; netstat -s; cat /proc/net/softnet_stat; } > $OUT/stats.txt 2>&1
dmesg -T | tail -300 > $OUT/dmesg.txt
journalctl -p warning --since "1 hour ago" --no-pager > $OUT/journal.txt 2>&1
iptables-save > $OUT/iptables.txt 2>&1; nft list ruleset > $OUT/nft.txt 2>&1
timeout 30 tcpdump -i any -nn -c 2000 -w $OUT/cap.pcap 2>/dev/null
tar czf $OUT.tgz -C /tmp $(basename $OUT); echo "수집 완료: $OUT.tgz"
```

E-3 지속 모니터링 한 줄 명령

판단 간헐적 장애 포착용

```
while true; do echo "$(date +%T) $(ping -c1 -W1 8.8.8.8 >/dev/null && echo up || echo DOWN)"; sleep 1; do
ne | tee ping.log
watch -n1 'ss -s; echo; nstat -az | egrep "Retrans|Drop"'
while sleep 5; do curl -o /dev/null -s -w "%{time_total} %{http_code}\n" http://<호스트>/; done
journalctl -f -p warning
ping -D -i 0.5 <IP> | awk '{print strftime("%T"), $0}' >> ping.log
```

부록 F. 증상 → 계층 빠른 판별표

증상	의심 계층	확인할 것
ethtool: Link detected: no	L1	케이블/포트/NIC, ethtool, dmesg
같은 대역인데 ping 불가, neigh FAILED/INCOMPLETE	L2	ip neigh, arping, VLAN, 스위치
ip neigh 에 중복 MAC / arping -D 응답	L2	IP·MAC 충돌, 브리지 루프
Network is unreachable	L3	ip addr, ip route, 서브넷·게이트웨이
ping 8.8.8.8 OK, ping 도메인 FAIL	L7	DNS: dig, resolv.conf, nsswitch
작은 패킷 OK, 큰 전송 중 멈춤	L2/L3	MTU: ping -M do -s, tracepath
즉시 Connection refused	L4/L7	ss -tlnp, systemctl status (서비스 다운)
30초 뒤 timed out	L3/L4	iptables DROP, 보안그룹, 경로, tcpdump
SYN 만 가고 SYN-ACK 없음	L3/L4	방화벽, 서버 다운, 경로
CLOSE_WAIT 수천 개	L4/L7	앱이 close() 미호출, 앱 로그/덤프
SYN_RECV 급증	L4	SYN flood, backlog, syncookies
nf_conntrack: table full	L3/L4	conntrack_max 증설, 타임아웃 조정
Cannot assign requested address	L4	포트 고갈, ip_local_port_range, tw_reuse
유휴 후 SSH/DB 연결 끊김	L5	keepalive, 방화벽/LB idle timeout
NFS stale file handle / 응답 없음	L5	rpcinfo, nfsstat, 마운트 옵션
SSL certificate problem (60)	L6	openssl s_client, 만료/체인/호스트명/시간
handshake failure	L6	프로토콜/암호군 불일치, SNI
한글이 ??? 로 표시	L6	locale, file -i, iconv
HTTP 502 Bad Gateway	L7	업스트림 다운, 프록시 로그
HTTP 504 Gateway Timeout	L7	업스트림 지연, proxy_read_timeout
HTTP 403	L7	권한/ACL/SELinux, namei -l
서비스 간헐적 느림	전계층	curl -w 분해, mtr, softnet_stat, 자원 병목
패킷 드롭 증가 (softnet dropped)	L1~L3	링버퍼, IRQ 분산, netdev_max_backlog

부록 G. 계층별 핵심 명령어 한 장 요약

계층	핵심 명령어
L1	`ip -br link | ethtool eth0 | ethtool -S eth0 | dmesg -T | grep -i link`
L2	`ip neigh | arping -I eth0 <IP> | ip -s link | tcpdump -e arp | bridge fdb`
L3	`ip addr | ip route get <IP> | ping | traceroute -n | mtr -rwzbc 100 | ping -M do -s`
L4	`ss -tulnp | ss -s | nc -zv <IP> <PORT> | iptables -L -n -v | nft list ruleset | nstat -az`
L5	`ss -o state established | ss -tnoi | sysctl net.ipv4.tcp_keepalive_* | nfsstat | who -a`

L6	`openssl s_client -connect H:443 | openssl x509 -checkend | locale | iconv | file -i`
L7	`curl -v / -w | dig +trace | systemctl status | journalctl -u | tail -f /var/log/...`
전체	`tcpdump -w cap.pcap | tshark -z conv,tcp | sar -n DEV,TCP | vmstat | iostat | dmesg`

진단 원칙

1. 증상 재현 및 범위 파악 (누가, 어디서, 언제부터, 무엇이 안 되는가)
2. L1→L7 순서로 정상/비정상 경계를 찾는다
3. 가설을 세우고 한 번에 하나만 변경한다
4. 변경 전 설정을 백업하고, 복구 방법을 먼저 정한다
5. 원인과 조치를 기록해 재발을 방지한다